

Information Technology University of the Punjab, Lahore

Analysis of Algorithms

Assignment # 3

Name : _____

Roll No: _____

Deadline: 6 May, 2026

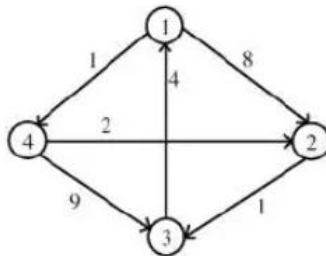
Maximum Marks: 100

Question 1:

Explain the core idea behind the Floyd-Warshall algorithm. How does it use an intermediate vertex k to progressively find shortest paths between all pairs of vertices? State the recurrence relation it uses and describe when the algorithm detects a negative-weight cycle.

Question 2:

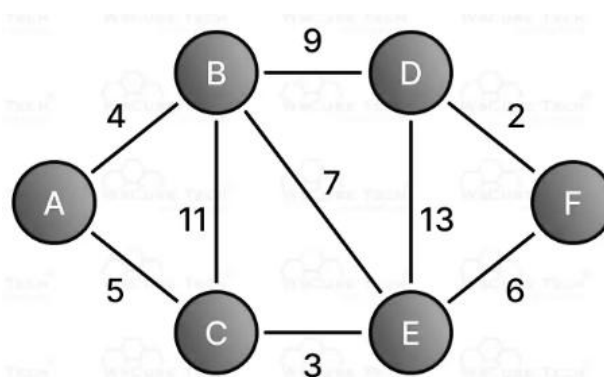
Use the Floyd-Warshall algorithm and find shortest path between all pair of vertices for the following graph.



Question 3:

Shortest Path using Dijkstra's Algorithm

Consider the following weighted graph. Using **Dijkstra's Algorithm**, find the shortest distance from source vertex **A** to every other vertex. Show all steps clearly and draw the final shortest path tree.



Question 4:

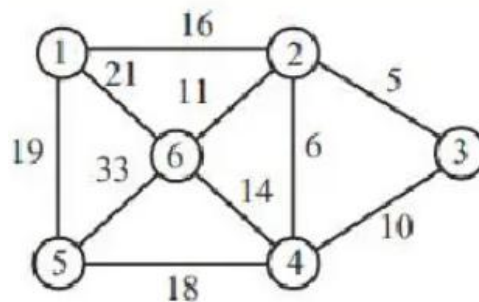
Explain how Dijkstra's Algorithm uses a greedy approach to find the shortest path from a single source. Why does it require all edge weights to be non-negative? Describe the role of the priority queue (min-heap) in the algorithm and how it ensures each vertex is processed in order of its current shortest distance. Explain the pseudocode of Dijkstra's Algorithm.

Question 5:

Explain the principle of Huffman Coding and why it produces an optimal prefix-free binary code. Describe how the min-heap (priority queue) is used during tree construction. How does the resulting code achieve data compression — provide a brief example comparing fixed-length vs. Huffman encoding. Also write pseudocode of Huffman Coding.

Question 6:

Find the shortest path for vertex 1 to vertex 3 in the following graph using Dijkstra's greedy algorithm.

**Question 7:**

Build the Huffman tree for the following character frequencies:

Character	A	B	C	D	E	F	G	H
Frequency	5	9	12	13	24	19	16	45

- Show the step-by-step merging of nodes to build the Huffman tree.
- Assign binary codes (0 = left, 1 = right) to each character.
- Calculate the total number of bits required to encode a message of 100 characters using these frequencies.

Question 8:

Given the following jobs (each takes 1 unit of time), find the optimal sequence that maximises total profit. Assume each unit of time is one slot (slots: 1, 2, 3).

Job	Deadline	Profit (\$)
J1	2	100
J2	1	50
J3	2	10
J4	1	20
J5	3	60

Show: (a) The sorted order of jobs by profit, (b) Slot allocation step-by-step, (c) Final scheduled jobs and total profit achieved.

Question 9:

Describe the difference between the 0/1 Knapsack Problem and the Fractional Knapsack Problem. Why can greedy algorithms solve the fractional version optimally but not the 0/1 version? Explain how dynamic programming solves the 0/1 Knapsack using a 2-D table.

Question 10:

A thief has a knapsack with a weight capacity of 8 kg. The available items are:

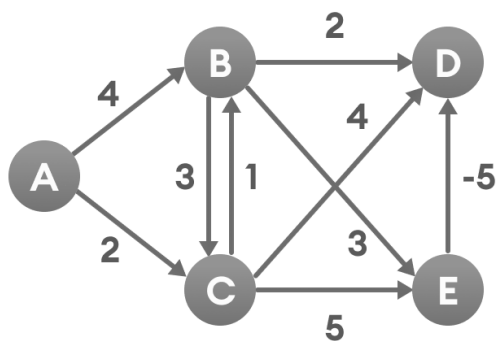
Item	Weight (kg)	Value (\$)
A	2	40
B	3	50
C	4	60
D	5	70

Build the complete DP table $dp[i][w]$ where i = item index and w = capacity. Determine the maximum value achievable and list the items selected.

Hint: $dp[i][w] = \max(dp[i-1][w], dp[i-1][w - \text{weight}[i]] + \text{value}[i])$ if $\text{weight}[i] \leq w$.

Question 11:

The provided graph contains a negative edge weight between nodes **E** and **D**. Explain why Dijkstra's algorithm might fail here and then apply the **Bellman-Ford algorithm** starting from **Vertex A** to determine the shortest path to **Vertex D**.



Question 12:

Explain why the Traveling Salesman Problem (TSP) is classified as NP-Hard. Describe the Dynamic Programming (Held-Karp) approach to solving TSP. What is the time complexity of the DP solution compared to brute force, and what does the bitmask represent in the DP state?

Question 13:

Examine the provided graph for a **Traveling Salesman Problem**.

1. Determine if a complete Hamiltonian cycle exists given the one-way (directed) nature of the paths.
2. If a cycle exists starting and ending at **Node A**, calculate the minimum travel cost.
3. If no cycle exists that visits every node exactly once, explain which node(s) create the bottleneck.

